2025/2026	AP – Projet 3 – Modération – Itération 3
BTS SIO 2SLAM	Auteur : PORTOLLEAU Anaïs & DELAUNAY-GUITTON Benjamin & CHARBONNEAU Tom
	Date de rédaction : 02/03/2026

Projet Modération pour R3st0.fr

Table des matières

Projet Modération pour R3st0.fr.....1
Itération 3 – Poursuite du développement de l'application JAVA.....2
Ticket 1 : Filtrage des avis masqués.....3
Programmes : 3
Test fonctionnel : 8
Ticket 2 : Suppression d'un avis.....12
Programmes : 12
Test fonctionnel : 16
Ticket 3 - Déploiement de la base de données : 23
Adaptation du projet :.....23
Note importante (après la finalisation du ticket 4) : 23
PhpMyAdmin :.....24
Ticket 4 : Mise à disposition de l'exécutable 26
Hébergement client FTP :.....26
Exécution de l'application : 28
Ticket 5 : Rédaction des tickets pour l'équipe du site web 31

Itération 3 – Poursuite du développement de l'application JAVA

- **itération 3**
  - durée : 1 semaine.
  - date limite de remise du livrable : avant le **30/03/2026**

Lien du GitLab : URL de notre dépôt

Ticket 1 : Filtrage des avis masqués

Branche utilisée : iteration3_ticket1

Programmes :

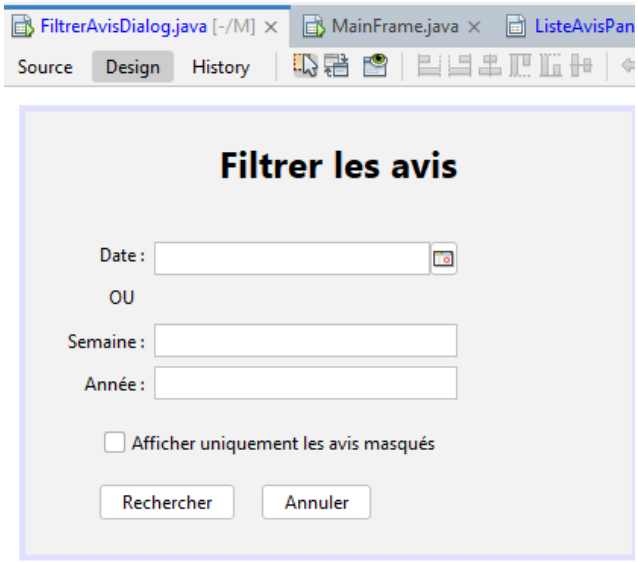
Tout d'abord, je me suis rendu dans le fichier *FiltrerAvisDialog.java* dans lequel j'ai ajouté la méthode canAccessMaskedAvis(). Cette méthode vérifie que l'utilisateur de la session n'est pas un modérateur.

```
31      /**
32       * Affiche la case "Afficher les avis masqués" si l'utilisateur est un
33       * responsable (manager).
34       */
35      private void canAccessMaskedAvis() {
36          if (!Session.isManager()) {
37              jCheckBoxAvisMasques.setVisible(false);
38          }
39      }
```

Puis j'exécute cette méthode dans le constructeur de la classe actuelle. De ce fait, à l'initialisation de la boite de dialogue, la méthode est exécutée.

```
24      public FiltrerAvisDialog(java.awt.Frame parent, boolean modal) {
25          super(parent, modal);
26          initComponents();
27          setTitle("Filtrer les avis");
28          canAccessMaskedAvis();
29      }
```

Ensuite, j'ai modifié le label de la case à cocher en rajoutant le mot « uniquement » afin de bien préciser l'action que permet ce bouton.



Enfin, je me suis rendu dans la classe *ListeAvisPanel.java*, où j'ai modifié deux méthodes appliquerFiltre() et chargerAvis() que j'ai découpé en 8 méthodes distinctes.

Pour rappel, cette méthode charge les avis dans le tableau, sans filtre d'imposé. Elle récupère toutes les critiques, initialise le tableau à vide, puis affiche uniquement les avis visibles.

```java
215     /**
216      * Charge les avis visibles dans le tableau (sans filtre).
217      */
218     private void chargerAvis() {
219         List<Critique> critiques = recupererCritiques(); // Récupération des données
220         initialiserTable(); // Initialisation de l'affichage
221
222         // Filtrage : uniquement visibles (false = pas masqués)
223         afficherCritiques(critiques, false, null, null, null);
224     }
```

De même, pour rappel, cette méthode applique un filtre sur les avis. Elle commence par récupérer les critiques, initialise le tableau à vide, puis affiche uniquement les critiques qui correspondent aux critères (date, semaine/année et statut visible ou masqué).

```java
226     /**
227      * Applique un filtre sur les avis et met à jour le tableau.
228      *
229      * @param dateFiltre date précise (optionnelle)
230      * @param semaineFiltre semaine (optionnelle)
231      * @param anneeFiltre année (optionnelle)
232      * @param afficherMasques true = afficher masqués, false = visibles
233      */
234     private void appliquerFiltre(java.sql.Date dateFiltre, Integer semaineFiltre,
235             Integer anneeFiltre, boolean afficherMasques) {
236
237         // Récupération des données
238         List<Critique> critiques = recupererCritiques();
239
240         // Réinitialisation du tableau
241         initialiserTable();
242
243         // Affichage avec filtre
244         afficherCritiques(critiques, afficherMasques, dateFiltre, semaineFiltre, anneeFiltre);
245     }
```

Ensuite, j'ai également ajouté une méthode initialiserTable() qui initialise le tableau des critiques à vide. Elle récupère d'abord le modèle du tableau avec jTableListeAvis.getModel(), puis elle vide complètement son contenu en mettant le nombre de lignes à 0 avec setRowCount(0). Ensuite, elle réinitialise la liste listeCritiquesAffichees en créant une nouvelle ArrayList. Ainsi, le tableau et la liste associée repartent de zéro, ce qui permet de reconstruire proprement l'affichage à partir de nouvelles données ou de nouveaux filtres.

```java
257     /**
258      * Initialise le tableau en le vidant et en recréant la liste affichée.
259      */
260     private void initialiserTable() {
261         DefaultTableModel model = (DefaultTableModel) jTableListeAvis.getModel();
262         model.setRowCount(0);
263
264         listeCritiquesAffichees = new ArrayList<>();
265     }
```

Ensuite j'ai ajouté une méthode afficherCritiques() qui sert à trier, filtrer et afficher une liste de critiques dans un tableau.

D'abord, elle reçoit une liste complète de critiques ainsi que plusieurs filtres : le statut (masqué ou non), une date précise, ou encore une semaine et une année. Elle récupère ensuite le modèle du tableau pour pouvoir ajouter des lignes à l'affichage.

Elle prépare aussi des outils pour gérer les dates, comme un formateur de date et un objet Calendar.

Puis, elle parcourt chaque critique une par une. Pour chaque critique, elle vérifie d'abord si elle possède une date de création. Si ce n'est pas le cas, elle l'ignore.
Puis, elle applique un premier filtre concernant le statut (visible ou masqué) grâce à la méthode correspondStatut. Si la critique ne correspond pas au filtre, elle est ignorée.

Par la suite, elle applique un second filtre sur la date via la méthode correspondDate(), qui vérifie si la critique correspond à la date, la semaine ou l'année demandée. Si la critique passe tous les filtres, elle est ajoutée dans le tableau avec ajouterLigneTable().
Enfin, elle est aussi ajoutée à une liste locale pour garder une trace des critiques affichées, ce qui permet de garder la cohérence entre les données affichées et les données manipulées.

```java
267     /** Parcourt les critiques, applique les filtres et les affiche dans le ...10 lines */
277     private void afficherCritiques(List<Critique> critiques, boolean afficherMasques,
278             java.sql.Date dateFiltre, Integer semaineFiltre, Integer anneeFiltre) {
279
280         DefaultTableModel model = (DefaultTableModel) jTableListeAvis.getModel();
281
282         SimpleDateFormat formateur = new SimpleDateFormat("yyyy-MM-dd");
283         Calendar calendrier = Calendar.getInstance();
284
285         for (Critique c : critiques) {
286             // Ignore si pas de date
287             if (c.getCreatedDate() == null) {
288                 continue;
289             }
290
291             // Filtre statut (visible / masqué)
292             if (!correspondStatut(c, afficherMasques)) {
293                 continue;
294             }
295
296             // Filtre date / semaine
297             if (!correspondDate(c, dateFiltre, semaineFiltre, anneeFiltre, formateur, calendrier)) {
298                 continue;
299             }
300
301             ajouterLigneTable(model, c); // Ajout dans la table
302             listeCritiquesAffichees.add(c); // Synchronisation liste affichée
303         }
304     }
```

Ensuite j'ai ajouté une méthode correspondStatut() qui permet de vérifier si une critique correspond au filtre de statut, c'est-à-dire si elle est masquée ou visible. Elle prend en paramètre une critique c ainsi qu'un booléen afficherMasques qui indique le type de critiques à afficher. Si afficherMasques est à true, cela signifie que l'on souhaite afficher uniquement les critiques masquées, donc la méthode retourne la valeur de c.isMasquer(). À l'inverse, si afficherMasques est à false, on souhaite afficher uniquement les critiques visibles, donc la méthode retourne !c.isMasquer(). Ainsi, cette méthode permet de filtrer facilement les critiques selon leur statut en retournant true uniquement si la critique correspond au filtre choisi.

```java
    /**
     * Vérifie si une critique correspond au filtre de statut.
     *
     * @param c critique
     * @param afficherMasques true = masqués, false = visibles
     * @return true si correspond
     */
    private boolean correspondStatut(Critique c, boolean afficherMasques) {
        return afficherMasques ? c.isMasquer() : !c.isMasquer();
    }
```

Ensuite cette méthode nommé correspondDate() qui sert à vérifier si une critique correspond à un filtre basé sur la date, la semaine ou l'année. Elle prend en paramètre la critique, les filtres éventuels, ainsi que des outils pour formater et manipuler les dates. Si une date précise est fournie, la méthode compare la date de création de la critique avec cette date en les formatant de la même manière, et elle retourne true si elles sont identiques. Si aucune date n'est donnée mais qu'une semaine et une année sont spécifiées, elle récupère la date de la critique, en extrait l'année et le numéro de semaine, puis vérifie qu'ils correspondent aux valeurs du filtre. Si aucune condition de filtrage n'est renseignée, la méthode retourne true, ce qui signifie que la critique est acceptée sans restriction.

```java
    /**
     * Vérifie si une critique correspond au filtre de date.
     */
    private boolean correspondDate(Critique c,
            java.sql.Date dateFiltre,
            Integer semaineFiltre,
            Integer anneeFiltre,
            SimpleDateFormat formateur,
            Calendar calendrier) {

        // Filtre par date
        if (dateFiltre != null) {
            String dateAvis = formateur.format(c.getCreatedDate());
            String dateFiltreStr = formateur.format(dateFiltre);
            return dateAvis.equals(dateFiltreStr);
        }

        // Filtre semaine + année
        if (semaineFiltre != null && anneeFiltre != null) {
            calendrier.setTime(c.getCreatedDate());

            int anneeAvis = calendrier.get(Calendar.YEAR);
            int semaineAvis = calendrier.get(Calendar.WEEK_OF_YEAR);

            return (anneeAvis == anneeFiltre && semaineAvis == semaineFiltre);
        }

        // Aucun filtre
        return true;
    }
```

Puis une méthode ajouterLigneTable() qui permet d'ajouter une critique dans un tableau graphique. Elle reçoit en paramètre le modèle du tableau ainsi que la critique à afficher. Elle commence par déterminer le statut de la critique : si c.isMasquer() est vrai, alors le statut sera « Masqué », sinon il sera « Visible ». Ensuite, elle crée un tableau d'objets contenant les différentes informations de la critique, comme la date de création, le pseudo de l'utilisateur, le nom du restaurant, le commentaire et le statut. Enfin, elle ajoute cette ligne au modèle du tableau avec model.addRow(row), ce qui permet d'afficher la critique dans l'interface.

```
348     /**
349      * Ajoute une critique dans le tableau.
350      *
351      * @param model modèle du tableau
352      * @param c critique à afficher
353      */
354     private void ajouterLigneTable(DefaultTableModel model, Critique c) {
355         // Détermination du statut
356         String statut = c.isMasquer() ? "Masqué" : "Visible";
357
358         Object[] row = {
359             c.getCreatedDate(),
360             c.getPseudoUtilisateur(),
361             c.getNomRestaurant(),
362             c.getCommentaire(),
363             statut
364         };
365
366         model.addRow(row);
367     }
```

Test fonctionnel :

Objectif : S'assurer que la case « Afficher uniquement les avis masqués » est seulement visible par le responsable.

Note importante :

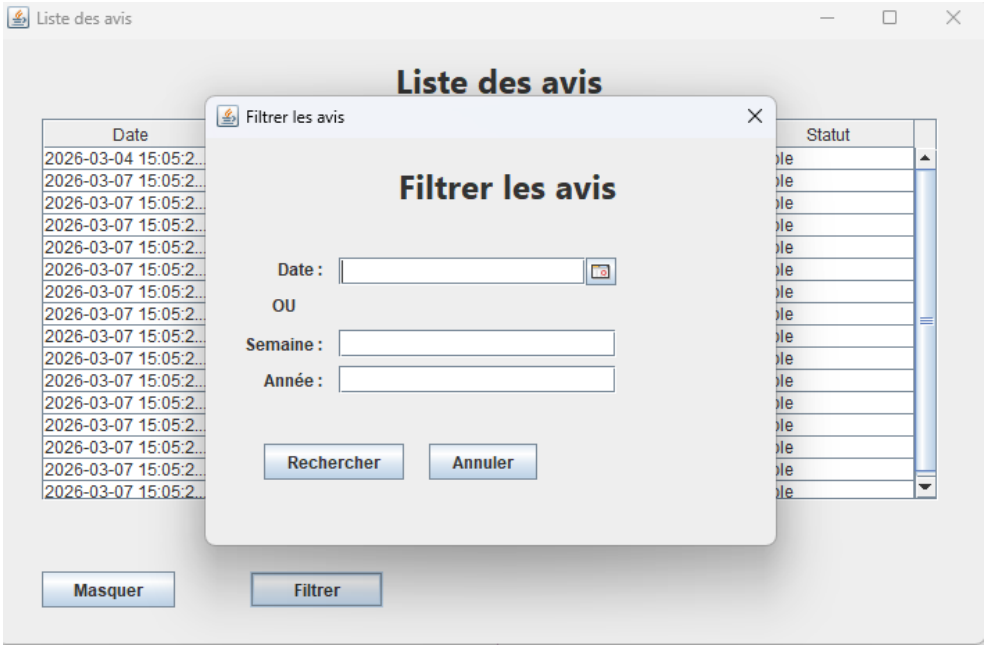
J'ai remarqué que si on n'ouvre pas le terminal de commande dans le répertoire où se situe le .env, il est impossible de se connecter à l'application. Cette erreur sera corrigée dans la branche « deploiement ». D'ici là, si vous continuez d'utiliser l'exécutable pour les tests, veuillez bien vous situer à la racine du projet pour exécuter le .JAR.

Cas modérateur :

Compte du modérateur :
Adresse email : user.moderator@test.com
Mot de passe : secret

1. On se connecte avec le compte du modérateur.

2. Lorsqu'on clique sur le bouton filtrer, la boite de dialogue apparaît sans la case « Afficher les avis masqués ».



Cas responsable :

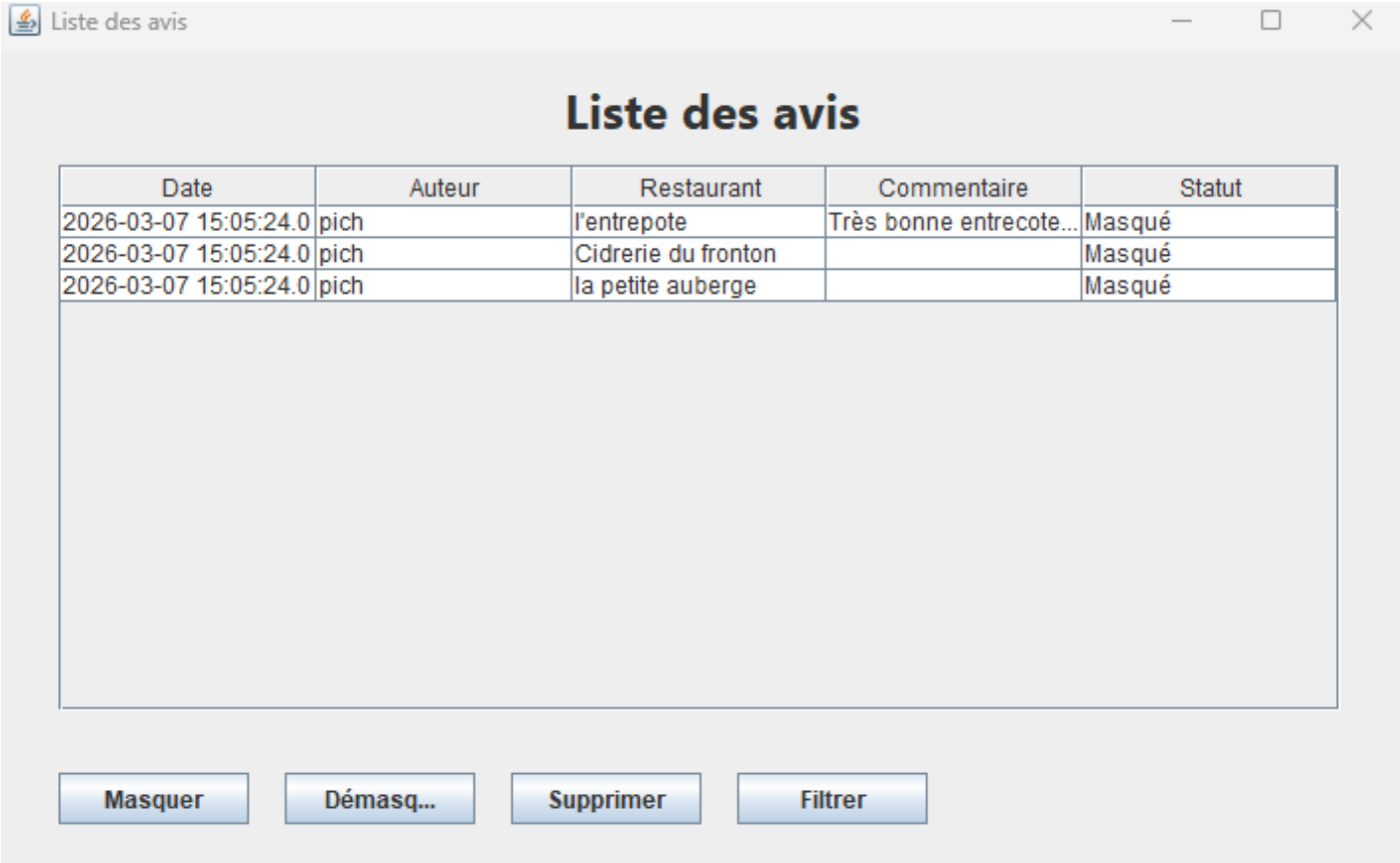
Compte du responsable :
Adresse email : user.manager@test.com
Mot de passe : secret

1. On se connecte avec les identifiants du compte du responsable.

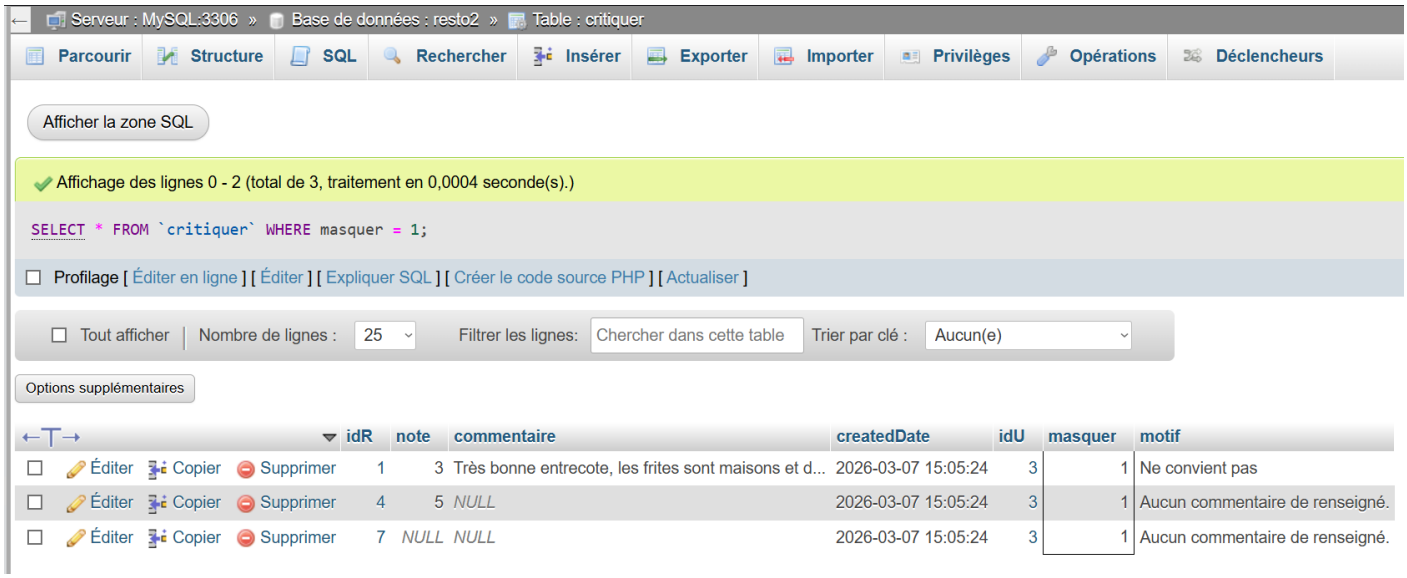
2. Lorsqu'on arrive sur l'écran « Liste des avis », on remarque seul les avis ayant le statut « visible » sont affichés dans le tableau.

3. Après avoir cliqué sur le bouton « Filtrer », cliquez sur la case « Afficher uniquement les avis masqués », puis validez en cliquant sur « Rechercher ».

4. Comme on peut le constater, seuls les avis masqués sont affichés.



5. En vérifiant dans la base de données, on remarque que ce sont les mêmes avis qui sont masqués. Il n'y a donc aucune



Ticket 2 : Suppression d'un avis

Branche utilisée : Iteration3_ticket2

Programmes :

J'ai ajouté dans CritiqueDao une méthode delete :

```
/**
 * Supprime définitivement une critique.
 *
 * @param idR identifiant du restaurant
 * @param idU identifiant de l'utilisateur
 * @return true si la suppression a réussi
 */
public boolean delete(int idR, int idU) {
    String sql = "DELETE FROM critiquer WHERE idR = ? AND idU = ?";

    try (Connection conn = ConnexionBDD.getConnexion();
         PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setInt(1, idR);
        ps.setInt(2, idU);

        return ps.executeUpdate() == 1;

    } catch (SQLException e) {
        System.out.println("Erreur delete critique : " + e.getMessage());
        return false;
    }
}
```

Cette méthode Java a pour objectif de supprimer définitivement une critique enregistrée dans une base de données. Elle prend en paramètres deux identifiants, celui du restaurant et celui de l'utilisateur, afin de cibler précisément la critique à supprimer. Une requête SQL de type DELETE est alors définie pour supprimer la ligne correspondante dans la table « critiquer ». Les points d'interrogation présents dans la requête servent de paramètres et permettent d'éviter les injections SQL, ce qui renforce la sécurité du programme. Ensuite, une connexion à la base de données est ouverte grâce à une méthode dédiée. Un objet PreparedStatement est utilisé pour préparer et exécuter la requête SQL de manière sécurisée. Les valeurs des paramètres idR et idU sont injectées dans la requête à l'aide des méthodes setInt. La requête est ensuite exécutée avec executeUpdate, qui retourne le nombre de lignes affectées par l'opération. Si une seule ligne est supprimée, cela signifie que la suppression a réussi et la méthode retourne true. Dans le cas contraire, la méthode retourne false. Enfin, si une erreur SQL se produit pendant l'exécution, celle-ci est capturée, un message d'erreur est affiché dans la console, et la méthode retourne false pour indiquer l'échec de l'opération.

J'ai ajouté également des tests pour cette méthode :

```java
// --- 8. Création + suppression ---
System.out.println("\n--- 8. Test de création puis suppression d'une critique ---");
System.out.println("\n--- Test création puis suppression ---");

Critique critiqueTestSupp = new Critique(
    (long) idR,
    (long) idU,
    5,
    "Test suppression",
    new Timestamp(System.currentTimeMillis()),
    false
);
dao.create(critiqueTestSupp);
System.out.println("Critique ajoutée pour le test.");

// --- Suppression ---
boolean resultatSuppr = dao.delete(idR, idU);

if (resultatSuppr) {
    System.out.println("Critique supprimée avec succès !");
} else {
    System.out.println("Échec de la suppression.");
}

// --- Vérification finale ---
System.out.println("\nVérification après suppression :");

Critique supprimee = dao.getByRestoId((long) idR)
    .stream()
    .filter(c -> c.getIdU() == idU)
    .findFirst()
    .orElse(null);

if (supprimee != null) {
    System.out.println("Erreur : la critique existe encore !");
} else {
    System.out.println("La critique a bien été supprimée.");
}
```

Ce test sert à vérifier le bon fonctionnement de la suppression d'une critique. D'abord, on crée un objet Critique avec des données de test (restaurant, utilisateur, note et commentaire).

Ensuite, cette critique est insérée en base de données avec la méthode create. Une fois la donnée ajoutée, on lance la suppression avec la méthode delete en utilisant l'identifiant du restaurant et de l'utilisateur.

Le résultat de la suppression est stocké dans une variable booléenne. Si la suppression réussit, un message de confirmation est affiché.

Sinon, un message indique que la suppression a échoué. Après cela, on vérifie que la critique a bien été supprimée en la recherchant dans la base.

On utilise getByRestoId puis un filtre sur l'identifiant utilisateur. Si la critique est encore trouvée, cela signifie qu'il y a une erreur. Sinon, cela confirme que la suppression a bien été effectuée. Ce test permet donc de valider à la fois la création et la suppression d'une critique. Il garantit que la base de données est correctement modifiée par les opérations du DAO.

Puis dans *ListeAvisPanel* j'ai ajouté une méthode supprimerAvis() :

CritiqueDao.java [-/M] × | ListeAvisPanel.java [-/M] × | TestCritiqueDao.java [-/M] ×

Source | Design | History

```java
    }

    /**
     * Méthode qui ma permettre à un reponsable
     * de supprimer un avis
     */
    private void supprimerAvis() {

        int idR = getSelectedIdR();
        int idU = getSelectedIdU();

        if (idR == -1 || idU == -1) {
            JOptionPane.showMessageDialog(this,
                    "Veuillez sélectionner un avis.",
                    "Erreur",
                    JOptionPane.ERROR_MESSAGE);
            return;
        }

        int confirmation = JOptionPane.showConfirmDialog(
                this,
                "Voulez-vous vraiment supprimer cet avis ?\nCette action est irréversible.",
                "Confirmation",
                JOptionPane.YES_NO_OPTION
        );

        if (confirmation != JOptionPane.YES_OPTION) {
            return;
        }

        CritiqueDao dao = new CritiqueDao();
        boolean ok = dao.delete(idR, idU);

        if (ok) {
            JOptionPane.showMessageDialog(this, "Avis supprimé avec succès !");
            refreshListeAvis();
        } else {
            JOptionPane.showMessageDialog(this,
                    "Erreur lors de la suppression.",
                    "Erreur",
                    JOptionPane.ERROR_MESSAGE);
        }
    }
```

Cette méthode permet à un responsable de supprimer un avis depuis l'interface graphique de l'application. Elle commence par récupérer les identifiants du restaurant et de l'utilisateur sélectionnés par l'utilisateur. Si aucun avis n'est sélectionné, c'est-à-dire si les identifiants valent -1, un message d'erreur s'affiche pour demander de sélectionner un avis. Dans ce cas, la méthode s'arrête immédiatement grâce à un return.

Ensuite, une boîte de dialogue de confirmation s'affiche pour éviter toute suppression accidentelle. Cette fenêtre demande à l'utilisateur s'il est sûr de vouloir supprimer l'avis, en précisant que l'action est irréversible. Si l'utilisateur choisit "Non", la méthode s'arrête sans effectuer aucune action.

Si l'utilisateur confirme la suppression, un objet CritiqueDao est créé afin d'accéder à la base de données. La méthode delete est alors appelée avec les identifiants pour supprimer l'avis correspondant. Le résultat de cette opération est stocké dans une variable booléenne.

Si la suppression réussit, un message de confirmation s'affiche et la liste des avis est actualisée grâce à la méthode refreshListeAvis. En revanche, si la suppression échoue, un message d'erreur est affiché pour informer l'utilisateur qu'un problème est survenu.

J' ai ajouté également un appel de cette méthode ci-dessus dans :

```
private void jButtonSupprimerActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    supprimerAvis();
}
```

Cette méthode correspond à l'action déclenchée lorsque l'utilisateur clique sur le bouton "Supprimer" dans l'interface graphique. Elle est automatiquement appelée par le gestionnaire d'événements lié au bouton. Le paramètre ActionEvent evt représente l'événement de clic, mais il n'est pas utilisé dans ce cas précis.

À l'intérieur de la méthode, une seule instruction est exécutée : l'appel à la méthode supprimerAvis(). Cela permet de séparer la logique de suppression de l'avis du code lié à l'interface graphique. Ainsi, lorsque l'utilisateur clique sur le bouton, la méthode supprimerAvis() est appelée et se charge de vérifier la sélection, demander une confirmation, puis effectuer la suppression dans la base de données.

Cette organisation rend le code plus clair, plus lisible et plus facile à maintenir. Elle permet également de réutiliser la méthode supprimerAvis() depuis d'autres parties du programme si nécessaire.

Test fonctionnel :

Test des méthodes :

Objectif : Le but de ce test est de vérifier le bon fonctionnement de la méthode de suppression et de vérifier si cela à bien été supprimé en base de données.

J'ai lancé d'abord le lancement de la vérification des tests de la méthode supprimer écrites ci-dessus et si on l'exécute cela donne ceci :

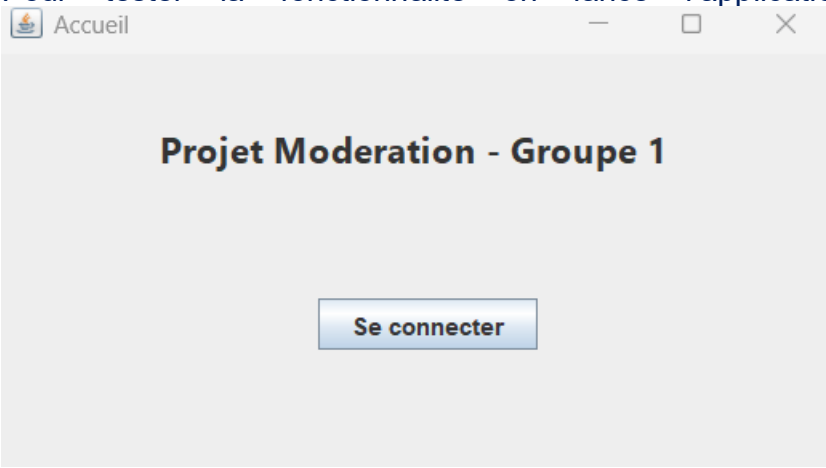
```
--- 8. Test de cr►ation puis suppression d'une critique ---

--- Test cr►ation puis suppression ---
Connexion BDD OK
java.sql.SQLIntegrityConstraintViolationException: Duplicata du champ '1-7' pour la clef 'critiquer.PRIMARY'
        at com.mysql.cj.jdbc.exceptions.SQLError.createSQLException(SQLError.java:118)
        at com.mysql.cj.jdbc.exceptions.SQLExceptionsMapping.translateException(SQLExceptionsMapping.java:122)
        at com.mysql.cj.jdbc.ClientPreparedStatement.executeInternal(ClientPreparedStatement.java:916)
        at com.mysql.cj.jdbc.ClientPreparedStatement.executeUpdateInternal(ClientPreparedStatement.java:1061)
        at com.mysql.cj.jdbc.ClientPreparedStatement.executeUpdateInternal(ClientPreparedStatement.java:1009)
        at com.mysql.cj.jdbc.ClientPreparedStatement.executeLargeUpdate(ClientPreparedStatement.java:1320)
        at com.mysql.cj.jdbc.ClientPreparedStatement.executeUpdate(ClientPreparedStatement.java:994)
        at modele.dao.CritiqueDao.create(CritiqueDao.java:33)
        at test.dao.TestCritiqueDao.main(TestCritiqueDao.java:112)
Critique ajout►e pour le test.
Connexion BDD OK
Critique supprim►e avec succ►s !

V►rification apr►s suppression :
Connexion BDD OK
La critique a bien ►t► supprim►e.
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  3.172 s
Finished at: 2026-03-23T16:12:46+01:00
------------------------------------------------------------------------
```

Le test commence par la création puis la suppression d'une critique. La connexion à la base de données est établie correctement. Une erreur apparaît lors de l'insertion, indiquant un doublon sur la clé primaire « 1-7 », ce qui signifie que la critique existe déjà en base. Malgré cette erreur, le programme continue son exécution. Le message indique ensuite que la critique a été ajoutée pour le test. Une nouvelle connexion à la base est effectuée. La suppression de la critique est ensuite réalisée avec succès. Un message confirme que la suppression a bien été effectuée. Une vérification finale est réalisée après suppression. La connexion à la base de données est de nouveau établie correctement. Le système confirme que la critique n'existe plus, ce qui valide la suppression. Le test se termine avec succès.

Test des interfaces :

Objectif : Le but de ces tests est de s'assurer du bon fonctionnement de la fonctionnalité suppression des avis.

Pour tester la fonctionnalité on lance l'application, et on arrive sur l'accueil :



Ensuite on se connecte et on va faire le test avec les deux comptes :

Test 1 :

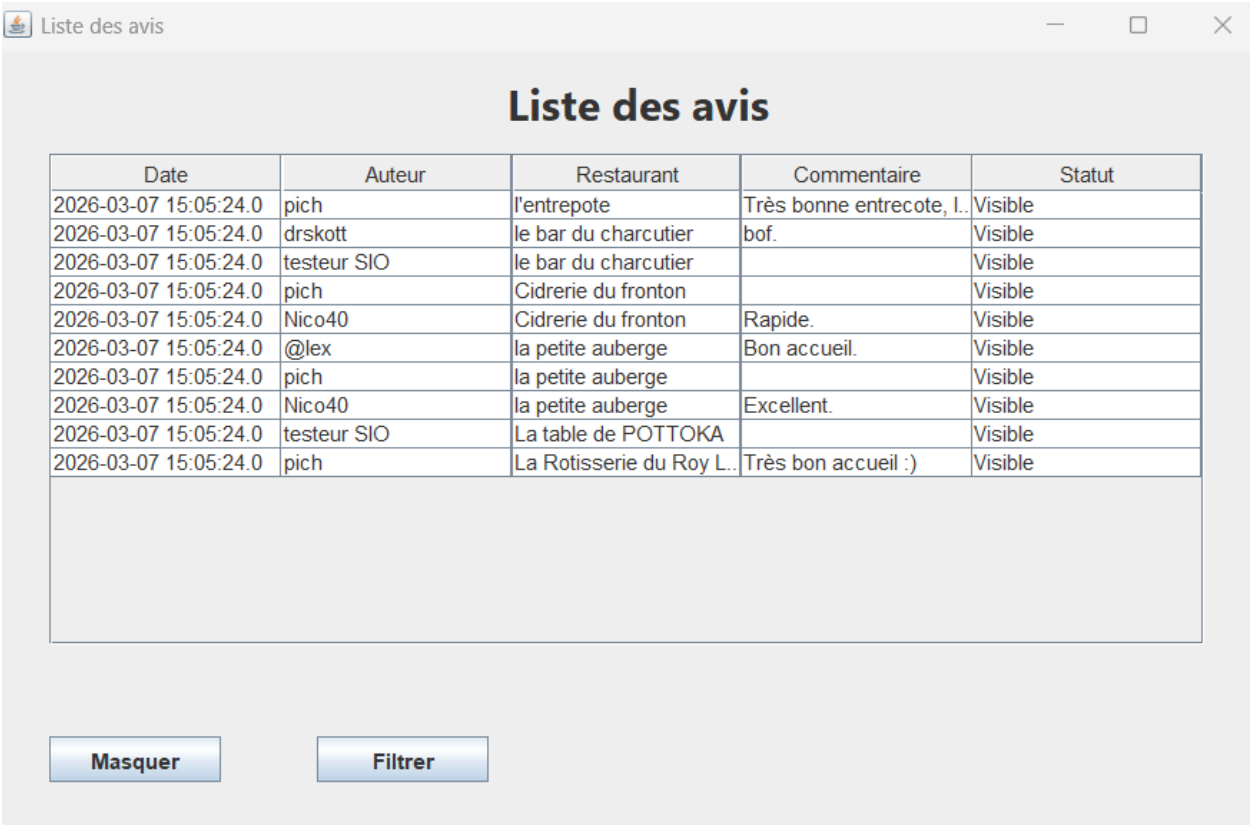
Objectif : S'assurer que le modérateur ne puisse pas supprimer d'avis

Cas modérateur :

Identifiant du compte modérateur :
Adresse email : user.moderator@test.com
Mot de passe : secret

Donc on rentre ces identifiants :

Puis on fait Valider pour se connecter et on arrive bien sur la liste des avis comme on peut le voir ci-dessous :



Date	Auteur	Restaurant	Commentaire	Statut
2026-03-07 15:05:24.0	pich	l'entrepote	Très bonne entrecote, l..	Visible
2026-03-07 15:05:24.0	drskott	le bar du charcutier	bof.	Visible
2026-03-07 15:05:24.0	testeur SIO	le bar du charcutier		Visible
2026-03-07 15:05:24.0	pich	Cidrerie du fronton		Visible
2026-03-07 15:05:24.0	Nico40	Cidrerie du fronton	Rapide.	Visible
2026-03-07 15:05:24.0	@lex	la petite auberge	Bon accueil.	Visible
2026-03-07 15:05:24.0	pich	la petite auberge		Visible
2026-03-07 15:05:24.0	Nico40	la petite auberge	Excellent.	Visible
2026-03-07 15:05:24.0	testeur SIO	La table de POTTOKA		Visible
2026-03-07 15:05:24.0	pich	La Rotisserie du Roy L..	Très bon accueil :)	Visible

On voit les avis qui sont que visibles (ticket fait dans itération précédente) et vu que c'est un modérateur il ne peut pas supprimer de ticket donc c'est bon. Le bouton Supprimer n'est pas visible à ce rôle donc normal qu'il ne puisse pas.

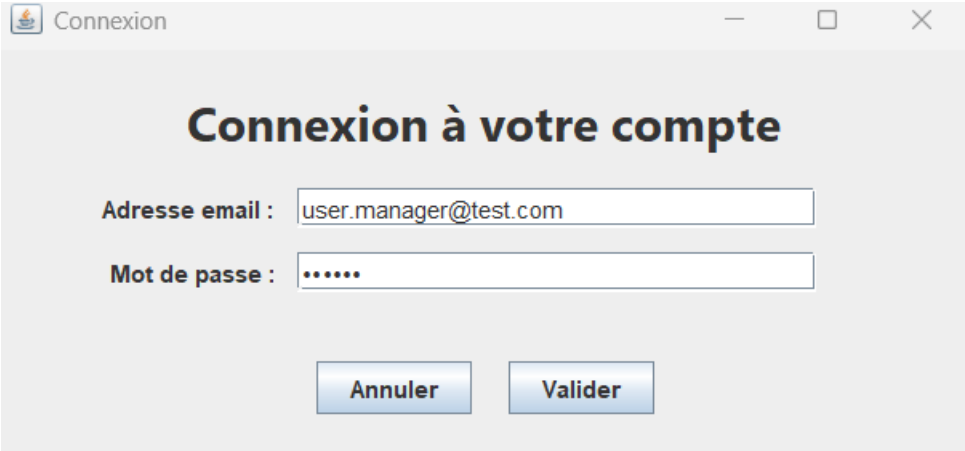
Cependant le manager peut donc on fait le test avec ce compte :

Test 2 :

Objectif : S'assurer que le manager puisse supprimer des avis et que cela s'enregistre en base :

Cas responsable :

Identifiant du compte manager :
Adresse email : user.manager@test.com
Mot de passe : secret

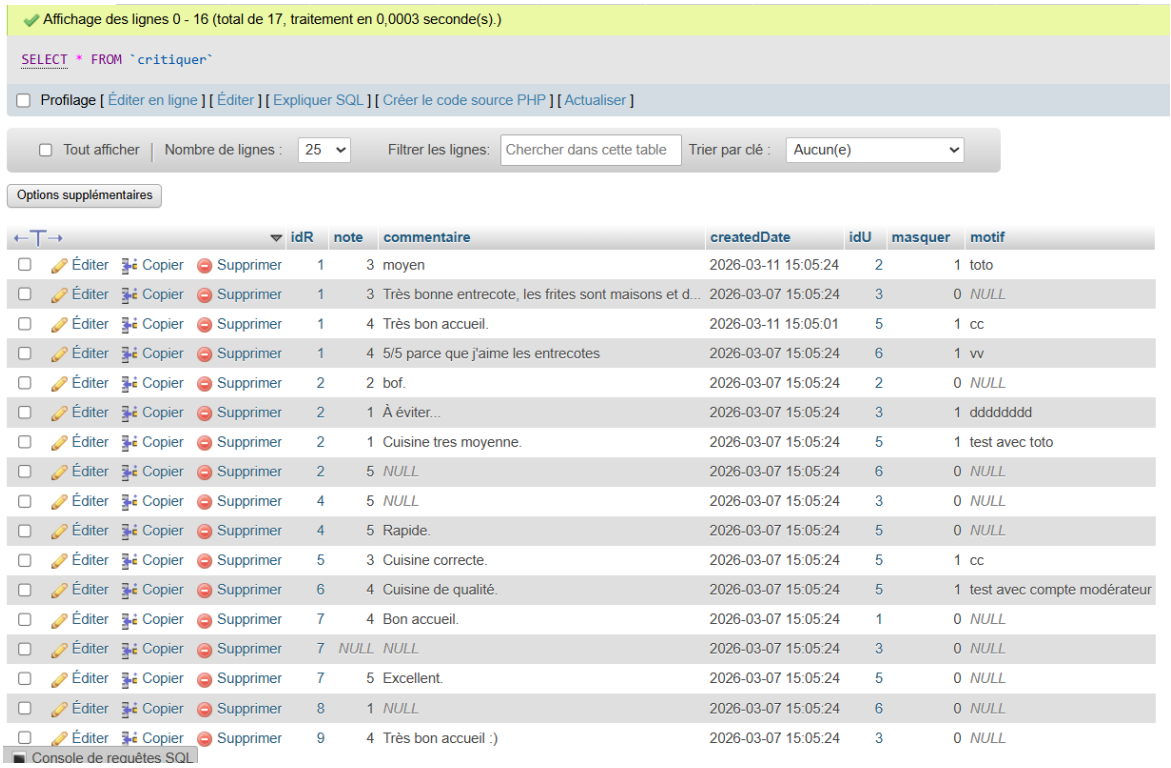


Puis on fait valider et on arrive sur la liste des avis :

Date	Auteur	Restaurant	Commentaire	Statut
2026-03-11 15:05:24.0	drskott	l'entrepote	moyen	Masqué
2026-03-07 15:05:24.0	pich	l'entrepote	Très bonne entrecote, l.	Visible
2026-03-11 15:05:01.0	Nico40	l'entrepote	Très bon accueil.	Masqué
2026-03-07 15:05:24.0	testeur SIO	l'entrepote	5/5 parce que j'aime le..	Masqué
2026-03-07 15:05:24.0	drskott	le bar du charcutier	bof.	Visible
2026-03-07 15:05:24.0	pich	le bar du charcutier	À éviter...	Masqué
2026-03-07 15:05:24.0	Nico40	le bar du charcutier	Cuisine tres moyenne.	Masqué
2026-03-07 15:05:24.0	testeur SIO	le bar du charcutier		Visible
2026-03-07 15:05:24.0	pich	Cidrerie du fronton		Visible
2026-03-07 15:05:24.0	Nico40	Cidrerie du fronton	Rapide.	Visible
2026-03-07 15:05:24.0	Nico40	Agadir	Cuisine correcte.	Masqué
2026-03-07 15:05:24.0	Nico40	Le Bistrot Sainte Cluq...	Cuisine de qualité.	Masqué
2026-03-07 15:05:24.0	@lex	la petite auberge	Bon accueil.	Visible
2026-03-07 15:05:24.0	pich	la petite auberge		Visible
2026-03-07 15:05:24.0	Nico40	la petite auberge	Excellent.	Visible
2026-03-07 15:05:24.0	testeur SIO	La table de POTTOKA		Visible

Si on veut tester la fonctionnalité supprimer pour les avis on a le bouton en bas de la fenêtre. Pour supprimer il faut d'abord sélectionner un avis puis cliquer sur supprimer.

Voici le visuel de la table avant le test :

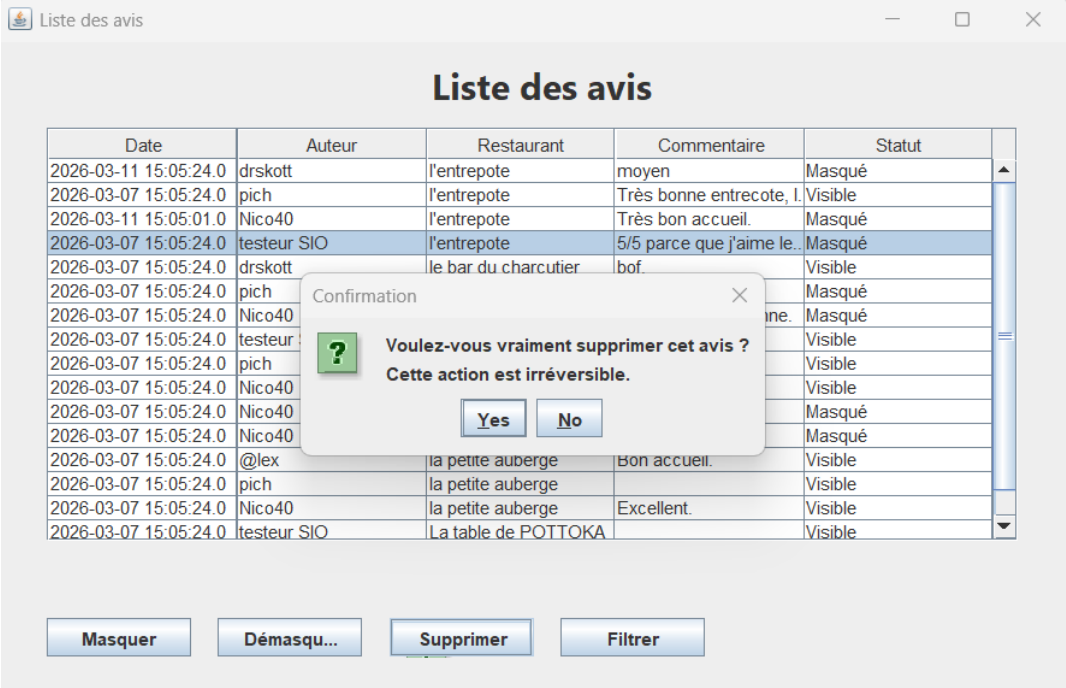


idR	note	commentaire	createdDate	idU	masquer	motif
1	3	moyen	2026-03-11 15:05:24	2	1	toto
1	3	Très bonne entrecote, les frites sont maisons et d...	2026-03-07 15:05:24	3	0	NULL
1	4	Très bon accueil.	2026-03-11 15:05:01	5	1	cc
1	4	5/5 parce que j'aime les entrecotes	2026-03-07 15:05:24	6	1	vv
2	2	bof.	2026-03-07 15:05:24	2	0	NULL
2	1	À éviter...	2026-03-07 15:05:24	3	1	ddddddd
2	1	Cuisine tres moyenne.	2026-03-07 15:05:24	5	1	test avec toto
2	5	NULL	2026-03-07 15:05:24	6	0	NULL
4	5	NULL	2026-03-07 15:05:24	3	0	NULL
4	5	Rapide.	2026-03-07 15:05:24	5	0	NULL
5	3	Cuisine correcte.	2026-03-07 15:05:24	5	1	cc
6	4	Cuisine de qualité.	2026-03-07 15:05:24	5	1	test avec compte modérateur
7	4	Bon accueil.	2026-03-07 15:05:24	1	0	NULL
7	NULL	NULL	2026-03-07 15:05:24	3	0	NULL
7	5	Excellent.	2026-03-07 15:05:24	5	0	NULL
8	1	NULL	2026-03-07 15:05:24	6	0	NULL
9	4	Très bon accueil :)	2026-03-07 15:05:24	3	0	NULL

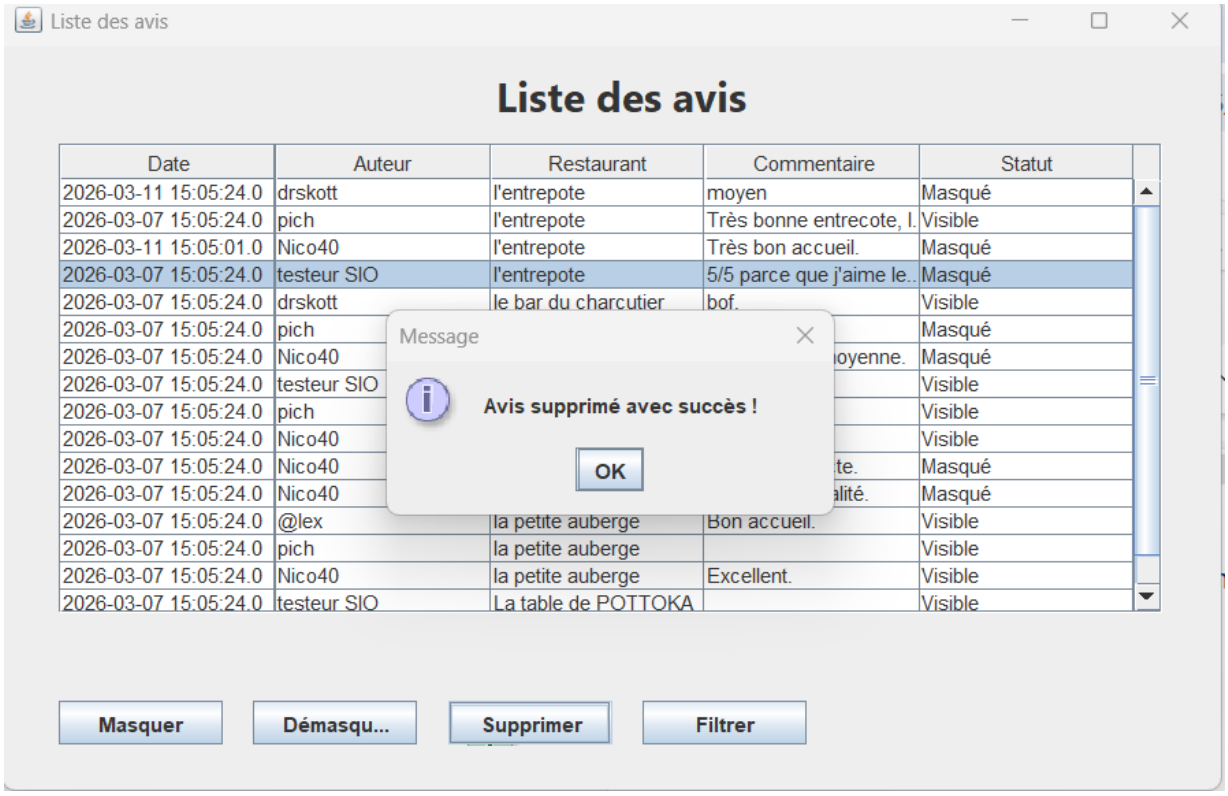
Puis quand on revient sur l'interface et qu'on fait sélectionne un avis :

Date	Auteur	Restaurant	Commentaire	Statut
2026-03-11 15:05:24.0	drskott	l'entrepote	moyen	Masqué
2026-03-07 15:05:24.0	pich	l'entrepote	Très bonne entrecote, l.	Visible
2026-03-11 15:05:01.0	Nico40	l'entrepote	Très bon accueil.	Masqué
2026-03-07 15:05:24.0	testeur SIO	l'entrepote	5/5 parce que j'aime le..	Masqué
2026-03-07 15:05:24.0	drskott	le bar du charcutier	bof.	Visible
2026-03-07 15:05:24.0	pich	le bar du charcutier	À éviter...	Masqué
2026-03-07 15:05:24.0	Nico40	le bar du charcutier	Cuisine tres moyenne.	Masqué
2026-03-07 15:05:24.0	testeur SIO	le bar du charcutier		Visible
2026-03-07 15:05:24.0	pich	Cidrerie du fronton		Visible
2026-03-07 15:05:24.0	Nico40	Cidrerie du fronton	Rapide.	Visible
2026-03-07 15:05:24.0	Nico40	Agadir	Cuisine correcte.	Masqué
2026-03-07 15:05:24.0	Nico40	Le Bistrot Sainte Cluq...	Cuisine de qualité.	Masqué
2026-03-07 15:05:24.0	@lex	la petite auberge	Bon accueil.	Visible
2026-03-07 15:05:24.0	pich	la petite auberge		Visible
2026-03-07 15:05:24.0	Nico40	la petite auberge	Excellent.	Visible
2026-03-07 15:05:24.0	testeur SIO	La table de POTTOKA		Visible

Puis on fait Supprimer et on a bien une demande de confirmation au cas où c'est une erreur :



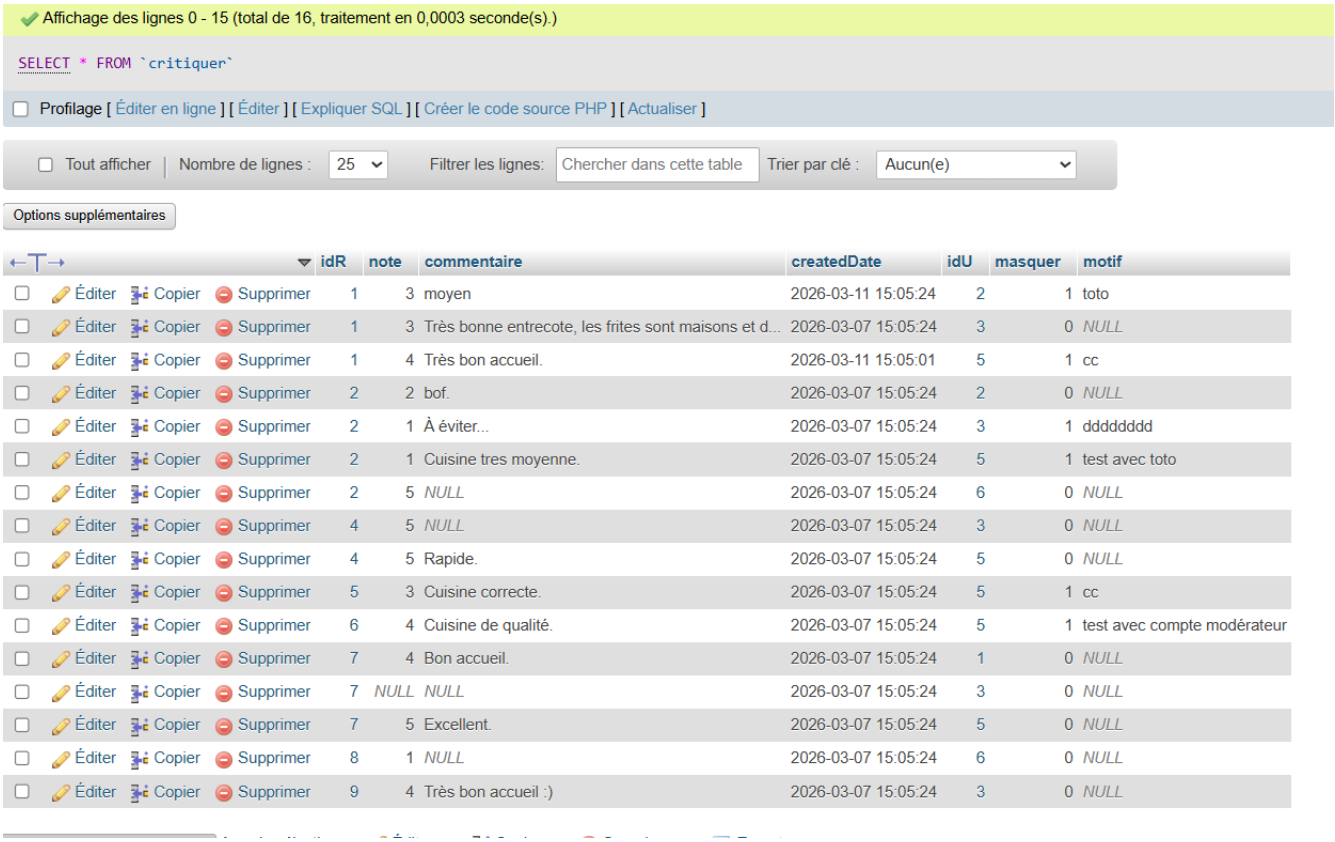
Puis on fait yes pour confirmer et on a une confirmation que l'avis a bien été supprimé :



Puis on fait ok et il a bien été supprimé de la liste comme on peut le voir ci-dessous :

Date	Auteur	Restaurant	Commentaire	Statut
2026-03-11 15:05:24.0	drskott	l'entrepote	moyen	Masqué
2026-03-07 15:05:24.0	pich	l'entrepote	Très bonne entrecote, l.	Visible
2026-03-11 15:05:01.0	Nico40	l'entrepote	Très bon accueil.	Masqué
2026-03-07 15:05:24.0	drskott	le bar du charcutier	bof.	Visible
2026-03-07 15:05:24.0	pich	le bar du charcutier	À éviter...	Masqué
2026-03-07 15:05:24.0	Nico40	le bar du charcutier	Cuisine tres moyenne.	Masqué
2026-03-07 15:05:24.0	testeur SIO	le bar du charcutier		Visible
2026-03-07 15:05:24.0	pich	Cidrerie du fronton		Visible
2026-03-07 15:05:24.0	Nico40	Cidrerie du fronton	Rapide.	Visible
2026-03-07 15:05:24.0	Nico40	Agadir	Cuisine correcte.	Masqué
2026-03-07 15:05:24.0	Nico40	Le Bistrot Sainte Cluq...	Cuisine de qualité.	Masqué
2026-03-07 15:05:24.0	@lex	la petite auberge	Bon accueil.	Visible
2026-03-07 15:05:24.0	pich	la petite auberge		Visible
2026-03-07 15:05:24.0	Nico40	la petite auberge	Excellent.	Visible
2026-03-07 15:05:24.0	testeur SIO	La table de POTTOKA		Visible
2026-03-07 15:05:24.0	pich	La Rotisserie du Roy ...	Très bon accueil :)	Visible

Et si on vérifie dans la table on peut voir que cela a bien été supprimé en base de données aussi:



←T→	idR	note	commentaire	createdDate	idU	masquer	motif
Éditer Copier Supprimer	1	3	moyen	2026-03-11 15:05:24	2	1	toto
Éditer Copier Supprimer	1	3	Très bonne entrecote, les frites sont maisons et d...	2026-03-07 15:05:24	3	0	NULL
Éditer Copier Supprimer	1	4	Très bon accueil.	2026-03-11 15:05:01	5	1	cc
Éditer Copier Supprimer	2	2	bof.	2026-03-07 15:05:24	2	0	NULL
Éditer Copier Supprimer	2	1	À éviter...	2026-03-07 15:05:24	3	1	ddddddd
Éditer Copier Supprimer	2	1	Cuisine tres moyenne.	2026-03-07 15:05:24	5	1	test avec toto
Éditer Copier Supprimer	2	5	NULL	2026-03-07 15:05:24	6	0	NULL
Éditer Copier Supprimer	4	5	NULL	2026-03-07 15:05:24	3	0	NULL
Éditer Copier Supprimer	4	5	Rapide.	2026-03-07 15:05:24	5	0	NULL
Éditer Copier Supprimer	5	3	Cuisine correcte.	2026-03-07 15:05:24	5	1	cc
Éditer Copier Supprimer	6	4	Cuisine de qualité.	2026-03-07 15:05:24	5	1	test avec compte modérateur
Éditer Copier Supprimer	7	4	Bon accueil.	2026-03-07 15:05:24	1	0	NULL
Éditer Copier Supprimer	7	NULL	NULL	2026-03-07 15:05:24	3	0	NULL
Éditer Copier Supprimer	7	5	Excellent.	2026-03-07 15:05:24	5	0	NULL
Éditer Copier Supprimer	8	1	NULL	2026-03-07 15:05:24	6	0	NULL
Éditer Copier Supprimer	9	4	Très bon accueil :)	2026-03-07 15:05:24	3	0	NULL

Ticket 3 - Déploiement de la base de données :

Adaptation du projet :

J'ai modifié le fichier restoJdbc.properties pour qu'il corresponde à l'utilisateur utilisé pour se connecter à PhpMyAdmin. Le mot de passe de l'utilisateur est hashé.

```
#####################################
# Paramètres de connexion JDBC
# Utilisation : décommenter les paramètres spécifiques au SGBD visé
# Les paramètres suivants doivent être renseignés : url, utilBD, mdpBD
#####################################

# MYSQL / MariaDB  local : mysql-connector-java-8.0.18.jar
url=jdbc:mysql://10.15.253.248:3306/2slamprj3eql_resto2
utilBD=2slamprj3eql
mdpBD=13x89nIC8/UHx2a872NS2A==
```

Note importante (après la finalisation du ticket 4) :

Après avoir déployer l'application, nous nous sommes rendu compte que si le JAR n'était pas dans le même répertoire que l'exécutable, l'application ne fonctionnerait pas correctement. De ce fait, nous avons pris la décision de remodifier le code de la classe *ConnexionBDD.java*, en ajoutant un nouvel attribut statique : SECRET_KEY qui contient la clé secrète de l'application anciennement située dans le *.env*. Nous avons conscience que stocker la clé de l'application de cette manière n'est pas très sécurisée. Toutefois cela reste plus sécurisé que de télécharger le *.env* en plus de l'exécutable.

```
18  public class ConnexionBDD {
19
20      /**
21       * Clé secrète de l'application. Elle est délibéremment stockée dans le
22       * fichier connexion puisque si elle reste dans le fichier ".env", alors il
23       * faudra aussi télécharger ce fichier pour exécuter le ".JAR". Il est donc
24       * plus sécurisé de la stocker ici, même si ce n'est pas la meilleure
25       * solution.
26       */
27      private static final String SECRET_KEY = "cleSuuuuperSecreteResto2ProjetAP";
```

Nous avons également pris le soin de supprimer lignes faisant référence aux objets Dotenv. Désormais, le code utilise la constante SECRET_KEY au lieu de l'ancien objet Dotenv.

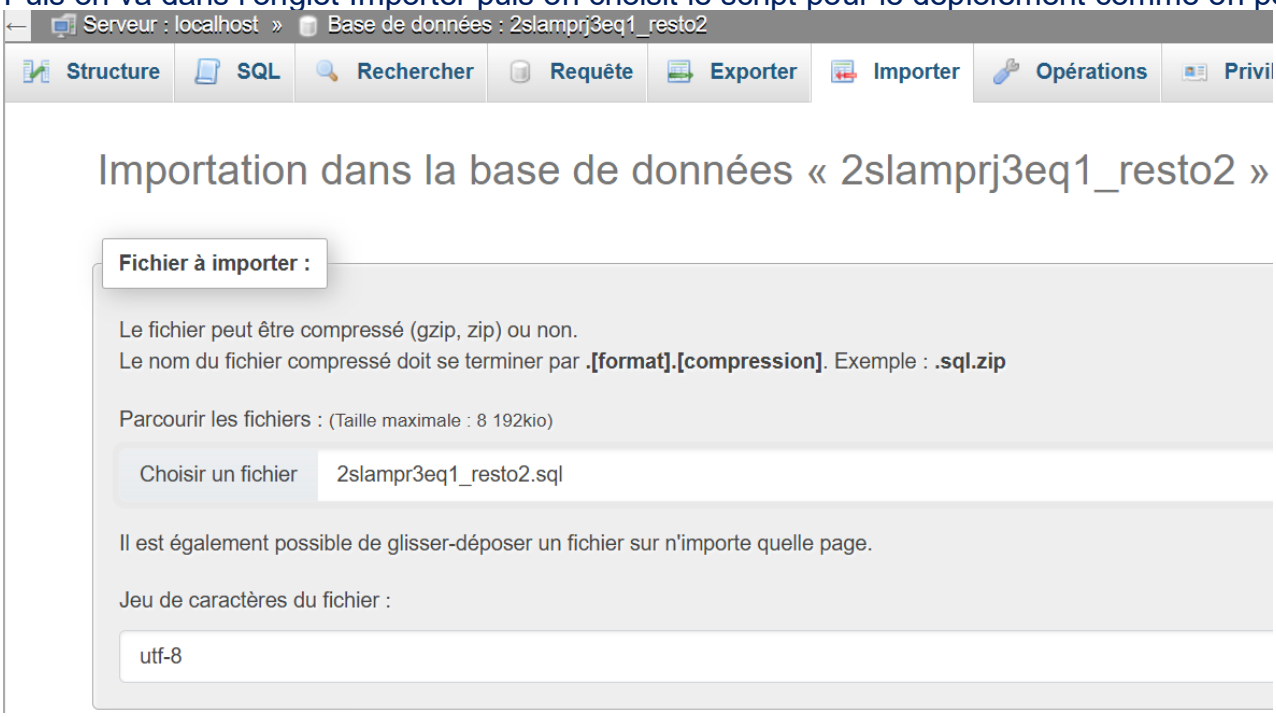
```
78          if (SECRET_KEY == null) {
79              throw new SQLException("Clé de chiffrement absente (constante SECRET_KEY)");
80          }
81
82          // Déchiffrement du mot de passe
83          String password = decrypt(encryptedPassword, SECRET_KEY);
```

PhpMyAdmin :

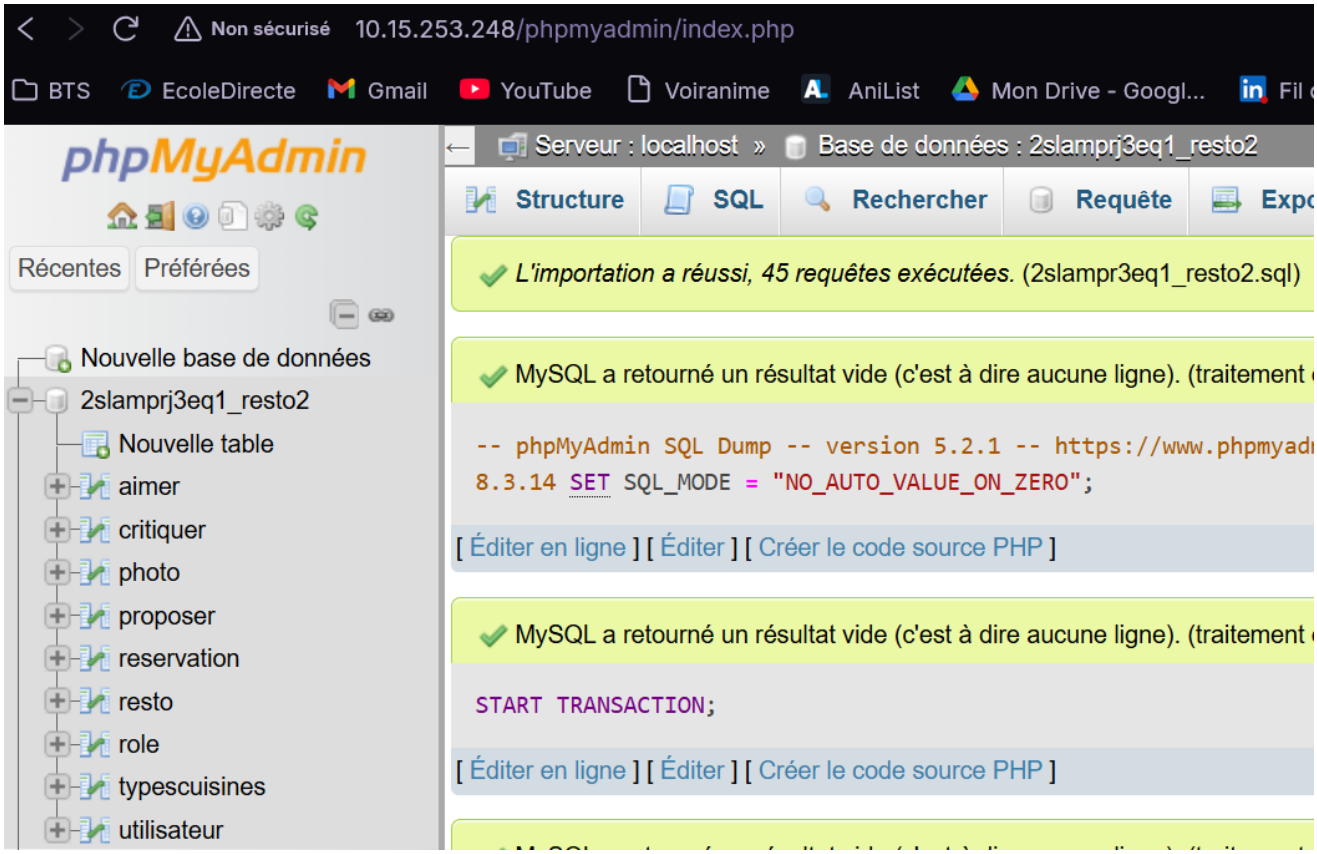
Puis je me suis connecté avec les identifiants de l'équipe pour déployer la base de données avec:
Identifiant : 2slamprj3eq1
Mot de passe (en clair) : Rz45=7?#N4

Puis on fait Connexion et on arrive sur l'interface PHPMyAdmin.

Puis on va dans l'onglet Importer puis on choisit le script pour le deploiement comme on peut voir:



Puis j'ai fait importer la base, et on peut voir ci-dessous que les tables se sont bien importées et le script d'exécution a bien fonctionné :



Ticket 4 : Mise à disposition de l'exécutable

Hébergement client FTP :

Les identifiants sont identiques à ceux utiliser pour se connecter à PhpMyAdmin.

Afin de rendre disponible l'application, je l'ai déployé sur le serveur de la Joliverie.

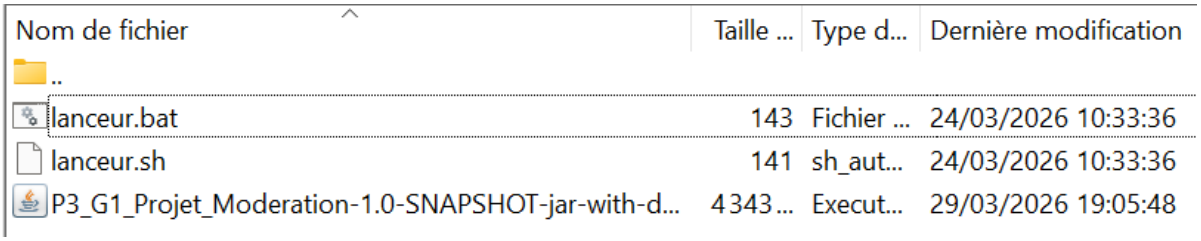
Pour commencer, j'ai utilisé FileZilla comme client ftp afin de transférer mon .jar et les 2 lanceurs (un pour Windows et un pour Linux en fonction de la machine utilisée).

Je commence par me connecter au serveur :

Ensuite j'ai créé un fichier "Java" dans notre répertoire :

Nom de fic...	Taill...	Type...	Derniè...	Droit...	Propr...
..					
.face.icon		Doss...	12/07/...	adfr...	2sla...
Java		Doss...	24/03/...	flcd...	2sla...
public		Doss...	02/03/...	flcd...	2sla...
.bash_lo...	220	Fichi...	23/04/...	adfr...	2sla...
.bashrc	3 526	Fichi...	23/04/...	adfr...	2sla...
.face	5 290	Fichi...	12/07/...	adfr...	2sla...
.profile	807	Fichi...	23/04/...	adfr...	2sla...

Dans celui-ci j'ai mis les différents fichiers nécessaires pour faire fonctionner l'application (.jar et les 2 lanceurs) dans le répertoire Java.



Nom de fichier	Taille ...	Type d...	Dernière modification
..			
lanceur.bat	143	Fichier ...	24/03/2026 10:33:36
lanceur.sh	141	sh_aut...	24/03/2026 10:33:36
P3_G1_Projet_Moderation-1.0-SNAPSHOT-jar-with-d...	4 343 ...	Execut...	29/03/2026 19:05:48

Pour télécharger les différents fichiers, on peut les récupérer directement via un navigateur web à ces différentes adresses :

Pour l'exécutable :
https://www3.jolsio.net/2slamprj3eq1/Java/P3_G1_Projet_Moderation-1.0-SNAPSHOT-jar-with-dependencies.jar

Pour éviter d'écrire la commande qui sert à exécuter l'application depuis dans un terminal, il est possible de télécharger les deux lanceurs suivants (choisir selon son système d'exploitation).

Les lanceurs :
https://www3.jolsio.net/2slamprj3eq1/Java/lanceur.sh (Lanceur Linux)
https://www3.jolsio.net/2slamprj3eq1/Java/lanceur.bat (Lanceur Windows)

Programmes dans les lanceurs :

lanceur.sh

C: > Users > benja > Downloads > SAVE_FILEZILLA_TOM > lanceur.sh

```
#!/bin/bash
echo "Lancement de l'application de moderation P3_G1..."
java -jar P3_G1_Projet_Moderation-1.0-SNAPSHOT-jar-with-dependencies.jar
```

lanceur.bat

Fichier Modifier Affichage

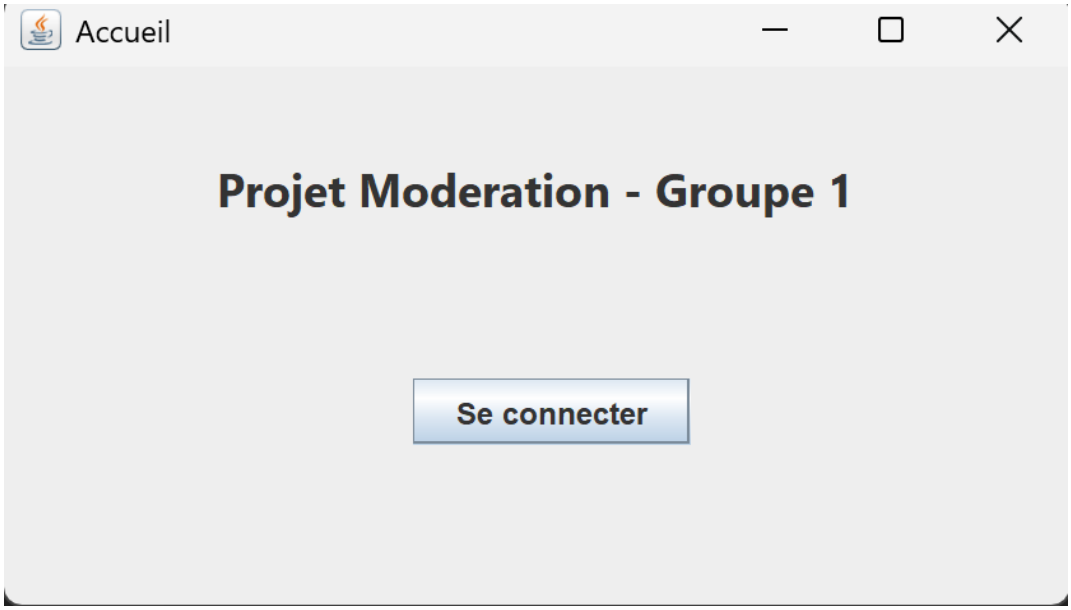
```
@echo off
echo Lancement de l'application de moderation P3_G1...
java -jar P3_G1_Projet_Moderation-1.0-SNAPSHOT-jar-with-dependencies.jar
pause
```

Si vous ne souhaitez pas utiliser les lanceurs, il suffit de recopier la commande suivante :
java -jar P3_G1_Projet_Moderation-1.0-SNAPSHOT-jar-with-dependencies.jar

Exécution de l'application :

Avant de commencer le test de l'exécutable, assurez vous d'être sur la connexion (SIO-LIC). Etant sur Windows je vais utiliser le lanceur.bat qui doit se situer dans le même répertoire que celui de l'exécutable :

L'application se lance correctement :



Ensuite je rentre les identifiants du modérateur par exemple qui sont (user.moderator@test.com / secret) :

Comme on peut voir on récupère bien les avis et on voit que les boutons masquer et filtrer car il n'a que ces droits là:

Date	Auteur	Restaurant	Commentaire	Statut
2026-03-04 15:05:24.0	drskott	l'entrepote	moyen	Visible
2026-03-07 15:05:24.0	Nico40	l'entrepote	Très bon accueil.	Visible
2026-03-07 15:05:24.0	testeur SIO	l'entrepote	5/5 parce que j'aim...	Visible
2026-03-07 15:05:24.0	yann	l'entrepote		Visible
2026-03-07 15:05:24.0	drskott	le bar du charcutier	bof.	Visible
2026-03-07 15:05:24.0	pich	le bar du charcutier	À éviter...	Visible
2026-03-07 15:05:24.0	Nico40	le bar du charcutier	Cuisine tres moyenne.	Visible
2026-03-07 15:05:24.0	testeur SIO	le bar du charcutier		Visible
2026-03-07 15:05:24.0	Nico40	Cidrerie du fronton	Rapide.	Visible
2026-03-07 15:05:24.0	Nico40	Agadir	Cuisine correcte.	Visible
2026-03-07 15:05:24.0	Nico40	Le Bistrot Sainte Cl...	Cuisine de qualité.	Visible
2026-03-07 15:05:24.0	@lex	la petite auberge	Bon accueil.	Visible
2026-03-07 15:05:24.0	Nico40	la petite auberge	Excellent.	Visible
2026-03-07 15:05:24.0	testeur SIO	La table de POTTOKA		Visible
2026-03-07 15:05:24.0	yann	La table de POTTOKA		Visible
2026-03-07 15:05:24.0	pich	La Rotisserie du R...	Très bon accueil :)	Visible

Ensuite je fais de même pour mon second utilisateur (user.manager@test.com / secret) :

Il fonctionne correctement et on peut voir que les droits sont également bons, le manageur peut masquer / démasquer un avis mais également filtrer les avis qu'il soit masqué ou non.

Date	Auteur	Restaurant	Commentaire	Statut
2026-03-04 15:05:24.0	drskott	l'entrepote	moyen	Visible
2026-03-07 15:05:24.0	Nico40	l'entrepote	Très bon accueil.	Visible
2026-03-07 15:05:24.0	testeur SIO	l'entrepote	5/5 parce que j'aim...	Visible
2026-03-07 15:05:24.0	yann	l'entrepote		Visible
2026-03-07 15:05:24.0	drskott	le bar du charcutier	bof.	Visible
2026-03-07 15:05:24.0	pich	le bar du charcutier	À éviter...	Visible
2026-03-07 15:05:24.0	Nico40	le bar du charcutier	Cuisine tres moyenne.	Visible
2026-03-07 15:05:24.0	testeur SIO	le bar du charcutier		Visible
2026-03-07 15:05:24.0	Nico40	Cidrerie du fronton	Rapide.	Visible
2026-03-07 15:05:24.0	Nico40	Agadir	Cuisine correcte.	Visible
2026-03-07 15:05:24.0	Nico40	Le Bistrot Sainte Cl...	Cuisine de qualité.	Visible
2026-03-07 15:05:24.0	@lex	la petite auberge	Bon accueil.	Visible
2026-03-07 15:05:24.0	Nico40	la petite auberge	Excellent.	Visible
2026-03-07 15:05:24.0	testeur SIO	La table de POTTOKA		Visible
2026-03-07 15:05:24.0	yann	La table de POTTOKA		Visible
2026-03-07 15:05:24.0	pich	La Rotisserie du R...	Très bon accueil :)	Visible

Sinon le code reste disponible à cette branche, c'est à partir de ce code qu'on a généré l'exécutable : deploiement.

Si vous souhaitez tester en local, il faut prendre cette branche : iteration3_finale. N'oubliez pas d'importer la BDD sur votre SGBD MySQL ou MariaDB. Il faut impérativement que le port soit 3306 et que vous importez également l'utilisateur : « resto_util ». Tous les scripts SQL sont dans le package « sql » de la branche. Il vous suffit d'importer le fichier « resto2.sql » et « resto2_ticket-6_create_utilisateur.sql » pour la création de l'utilisateur.

Ticket 5 : Rédaction des tickets pour l'équipe du site web

J'ai rédigé les 2 tickets nécessaire pour l'équipe du site web.

Le premier concernant le changement de cryptage dans la bdd qui est passé ce md5 à bcrypt :

Passer le cryptage sur bcrypt des mots de passe du site web (anciennement md5)

#21

Le cryptage md5 ayant des failles, nous avons préféré utiliser bcrypt que md5, donc il faudrait faire le changement également sur le site web.

Et le second concernant l'affichage des avis sur le site web :

Ajouter les avis des différents restaurants directement sur le site web

#22

Etant donné que l'application de modération des avis est fonctionnelle il faut ajouter les avis sur le site web.